

EXAMEN · SEGUNDO PARCIAL

DESARROLLO WEB BACKEND

Construcción de un CRUD completo · Laravel 13 + PostgreSQL

Asignatura	Desarrollo Web Backend (INF560)
Carrera	Ingeniería Informática — UATF
Docente	M. Sc. Huáscar Fedor Gonzales Guzmán
Modalidad	Examen práctico individual (en computadora)
Tiempo	120 minutos
Puntaje total	100 puntos

Apellidos y Nombres:	Registro / C.I.:
Fecha:	Calificación:

1. Instrucciones generales

1. Lea íntegramente el enunciado antes de comenzar. La distribución del puntaje está al final del documento.
2. Trabaje sobre un proyecto Laravel 13 nuevo, con PHP 8.3 o superior y base de datos PostgreSQL.
3. El examen es individual. Está prohibido compartir código. Puede consultar únicamente la documentación oficial de Laravel.
4. Realice un commit en Git por cada parte resuelta (mensajes descriptivos). La ausencia de historial de Git penaliza la entrega.
5. La aplicación debe ejecutarse sin errores con `php artisan migrate --seed` y `php artisan serve`.

2. Contexto del problema

Una pequeña empresa comercial necesita un sistema web para administrar su catálogo de productos. El sistema debe permitir registrar las categorías a las que pertenecen los productos y gestionar el inventario de cada producto a través de una interfaz web completa.

Su tarea consiste en construir, con Laravel 13, una aplicación que implemente un CRUD completo (Crear, Leer, Actualizar y Eliminar) sobre el recurso Producto, respetando la relación que existe entre productos y categorías.

MODELO DE DATOS

El sistema maneja dos entidades relacionadas mediante una relación uno-a-muchos:

- Una **Categoría** agrupa muchos productos (**hasMany**).
- Un **Producto** pertenece a una sola categoría (**belongsTo**).

3. Requerimientos técnicos

Componente	Versión / Tecnología exigida
Framework	Laravel 13
Lenguaje	PHP 8.3 o superior
Base de datos	PostgreSQL
Vistas	Blade (server-side rendering)
Control de versiones	Git con commits por parte resuelta

4. Especificación de la base de datos

4.1 Tabla: categorias

Campo	Tipo	Restricciones
id	bigIncrements	Llave primaria
nombre	string(100)	Obligatorio, único
descripcion	text	Opcional (nullable)
activo	boolean	Por defecto true
timestamps	timestamps	created_at / updated_at

4.2 Tabla: productos

Campo	Tipo	Restricciones
id	bigIncrements	Llave primaria
categoria_id	foreignId	FK → categorias, onDelete cascade
nombre	string(150)	Obligatorio
sku	string(50)	Obligatorio, único
precio	decimal(10,2)	Obligatorio, ≥ 0
stock	integer	Obligatorio, ≥ 0
disponible	boolean	Por defecto true
timestamps	timestamps	created_at / updated_at

5. Tareas a desarrollar

Parte 1 — Migraciones (15 pts)

- Cree las migraciones de las tablas categorias y productos según la especificación de la sección 4.
- Defina correctamente la llave foránea categoria_id con su restricción de integridad (onDelete cascade).

- Respete el orden de creación: la tabla padre (categorias) debe migrarse antes que la tabla hija (productos).

Parte 2 — Modelos y relaciones (15 pts)

- Genere los modelos Eloquent Categoria y Producto.
- Defina el atributo `$fillable` en ambos modelos.
- Configure los `casts` apropiados (por ejemplo, precio a decimal y activo / disponible a boolean).
- Declare las relaciones: `Categoria::productos()` (hasMany) y `Producto::categoria()` (belongsTo).

Parte 3 — Factories y Seeders (10 pts)

- Cree factories para Categoria y Producto con datos coherentes (precios y stock realistas, SKU único).
- Implemente un seeder que genere al menos 5 categorías y 20 productos asociados a categorías existentes.
- La aplicación debe poblarse con `php artisan migrate --seed`.

Parte 4 — Rutas y controlador resource (20 pts)

- Genere un controlador de tipo resource para **Producto** e impleméntelo con los 7 métodos: `index`, `create`, `store`, `show`, `edit`, `update`, `destroy`.
- Registre la ruta con `Route::resource()` en `routes/web.php`.
- En `index` cargue los productos con su categoría usando *eager loading* (`with('categoria')`) para evitar el problema N+1.

Parte 5 — Validación con Form Request (15 pts)

- Cree un **Form Request** (por ejemplo `ProductoRequest`) y úselo en `store` y `update`.
- Las reglas deben validar: nombre requerido; sku requerido y único (ignorando el propio registro al actualizar); precio numérico ≥ 0 ; stock entero ≥ 0 ; `categoria_id` existente en la tabla `categorias`.
- Muestre los mensajes de error de validación en los formularios.

Parte 6 — Vistas Blade (20 pts)

- Cree un layout base reutilizable (`@extends` / `@yield`) y las cuatro vistas del CRUD: `index`, `create`, `edit`, `show`.
- `index`: tabla con nombre, SKU, categoría, precio, stock y acciones (ver, editar, eliminar).
- `create` / `edit`: formularios con `@csrf`, `@method('PUT')` donde corresponda y un `<select>` poblado con las categorías. Reutilice los campos con `old()`.

Parte 7 — Mensajes flash y confirmación (5 pts)

- Tras crear, actualizar o eliminar, muestre un mensaje flash de sesión (`session('success')`).
- La eliminación debe solicitar confirmación al usuario antes de ejecutarse.

6. Comandos de referencia (opcionales)

Puede apoyarse en los siguientes comandos de Artisan. Su uso no es obligatorio, pero agiliza el desarrollo:

```
# Crear modelo + migración + factory + seeder + controlador resource
php artisan make:model Categoria -mfs
php artisan make:model Producto -mfs --resource

# Form Request de validacion
php artisan make:request ProductoRequest

# Migrar y poblar la base de datos
php artisan migrate --seed

# Levantar el servidor de desarrollo
php artisan serve
```

7. Rúbrica de evaluación

Criterio evaluado	Detalle	Puntaje
Migraciones	Estructura, tipos de datos y llave foránea con integridad referencial	15
Modelos y relaciones	fillable, casts y relaciones hasMany / belongsTo correctas	15
Factories y Seeders	Datos coherentes, ≥ 5 categorías y ≥ 20 productos	10
Rutas y controlador	Resource con 7 métodos y eager loading (sin N+1)	20
Validación (Form Request)	Reglas completas y mensajes mostrados en las vistas	15
Vistas Blade	Layout + index/create/edit/show funcionales con old()	20
Mensajes flash y confirmación	Feedback al usuario y confirmación de borrado	5
TOTAL	Aplicación funcional ejecutable sin errores	100

8. Lista de verificación antes de entregar

- ☐ El proyecto usa Laravel 13 y conecta correctamente a PostgreSQL.
- ☐ `php artisan migrate --seed` se ejecuta sin errores y crea datos de prueba.
- ☐ Las dos migraciones y la llave foránea están definidas y en el orden correcto.
- ☐ Ambos modelos tienen fillable, casts y las relaciones declaradas.
- ☐ El controlador resource implementa los 7 métodos del CRUD.
- ☐ La validación se realiza mediante un Form Request reutilizado en store y update.
- ☐ Las vistas index, create, edit y show funcionan y muestran errores y mensajes flash.
- ☐ La eliminación pide confirmación.
- ☐ El historial de Git refleja commits por cada parte resuelta.
- ☐ El comprimido NO incluye `vendor/` ni `node_modules/` e incluye `.env.example`.

Fin del examen. Éxitos.